
Was ist Containervirtualisierung?

Unter Containervirtualisierung versteht man ein Konzept, bei dem interne Funktionen des Betriebssystems dazu genutzt werden, Anwendungen voneinander isoliert auf demselben Hostsystem zu betreiben.

Wie virtuelle Maschinen werden Container dazu verwendet, Anwendungen voneinander isoliert auf einem System zu betreiben. Das ermöglicht passgenaue Hosts und spart damit Ressourcen.

Im Gegensatz zur Voll- oder Paravirtualisierung bei virtuellen Maschinen muss dafür kein vollständiges und unabhängiges Betriebssystem emuliert werden. Stattdessen greift die Containervirtualisierung auf den Kernel des Hosts zu und kann dessen Betriebssystem nutzen.

Entwicklung der Containervirtualisierung

In ihren Grundzügen existiert die Containervirtualisierung schon seit Anfang der 80er Jahre. Ihre Ursprünge liegen in dem 1979 eingeführten Linux-Kommando chroot, das es erstmals erlaubte gewisse Teile des Betriebssystems untereinander zu entkoppeln. Da die Performance damaliger Systeme jedoch noch nicht annähernd so hoch war wie in der heutigen Zeit, blieb die Entwicklung sinnvoller Anwendungsszenarien des Befehls aus und die Verbreitung blieb gering.

Erst gegen Mitte der 2000er, als zum einen leistungstärkere Hardware und zum anderen Weiterentwicklungen der Virtualisierungsmethoden entwickelt wurden, gewann das Konzept an Bedeutung. So ermöglichten OpenVZ und das darauf aufbauende Virtuozzo es mithilfe der Virtualisierung erstmals, mehrere Webseiten auf einem einzigen Server zu hosten.

Mittlerweile existieren für Unternehmen ausgereifte [Lösungen wie Kubernetes](#), ein Tool zur Container-Orchestrierung, das ursprünglich von Google entwickelt wurde und mit dem auch komplexe IT-Infrastrukturen abgebildet werden können.

Moderne Implementierung als Namespaces und Cgroups

den Prozess vereinfachen, sondern auch minimieren die Größe des Zugriffs auf die benötigten Ressourcen.

Noch immer war der Zugriff auf diese Features nicht besonders nutzerfreundlich. Das änderte sich erst 2013 mit dem Auftritt von dotcloud. Mit dem heutzutage [allseits bekannten Docker](#) vereinfachten sie den Zugriff auf diese Techniken und verhalfen damit der Containervirtualisierung endgültig zum Durchbruch.

In der Folge entwickelte sich ein weitflächiges Ökosystem rund um Container, von Alternativen wie rkt, bis hin zu [großen Orchestrierungslösungen wie Kubernetes](#).

Konzepte/Techniken	Beschreibung
chroot (1979)	eine Funktion unter Unix-Systemen, um das Rootverzeichnis zu ändern
UML (User Mode Linux)	erlaubt, komplette Linux-Kernel als Anwendungsprozesse innerhalb operierender Linux-Systeme auszuführen
LXC (Linux Container)	virtuelle Umgebung, die ihre eigenen Prozesse besitzt, doch für diese gemeinschaftlich den Kernel des Hostsystems nutzt
rkt	Alternative zu Docker, mittlerweile eingestellt, entwickelt von CoreOS
Podman	Engine für das Management von Containern

Die Vorteile der Containervirtualisierung

Container sind oft um einiges kleiner als virtuelle Maschinen, da kein vollständiges Betriebssystem emuliert werden muss.

Aus dem vergleichsweise geringen Overhead leitet sich der größte Vorteil der Technologie ab: Die Größensparnis sorgt für schnellere Start- und Stoppzeiten und spart wertvollen Speicherplatz.

Darüber hinaus eröffnet die standardisierte Methodik eine hohe Portabilität zwischen verschiedenen Plattformen. In Container gekapselte Applikationen lassen sich leicht migrieren. Die Standardisierung ermöglicht das Teilen der Container über mehrere Entwicklerteams hinweg.

Bedenken richten sich in erster Linie an die Sicherheit so betriebener Workloads. Da sich die isolierten betriebenen Anwendungen denselben Kernel teilen, besteht potenziell die Gefahr aus einem Container auszubrechen und auf Root und andere Applikationen zuzugreifen. Systembibliotheken werden weiterhin nicht geteilt, sodass Container unabhängig voneinander aktualisiert werden müssen.

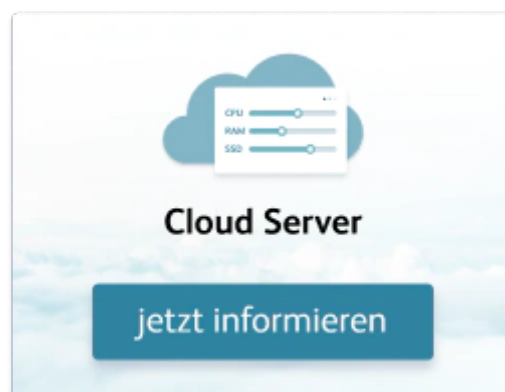
Doch vor allem im Bereich Sicherheit wird viel geforscht und entwickelt. Mit [SELinux](#) steht ein Framework bereit, das Container sicherer macht. Zusätzliche Absicherung erhält man, wenn man Container mit möglichst wenig Rechten ausstattet und betreibt. Auch [Seccomp](#), eine Sicherheitserweiterung des Linux-Kernels, soll mithelfen, Container weiter abzusichern.

Kubernetes bietet darüber hinaus rollenbasierte Zugriffssteuerung (Role-based Access Control, RBAC) an. Damit lassen sich detaillierte und spezifische Berechtigungssätze konfigurieren, mit denen sich die Interaktion von Nutzern mit Objekten limitieren lässt.

Containervirtualisierung in Produktivumgebungen?

Produktivumgebungen wechseln zunehmend von On-Premise-Umgebungen in die Cloud. Dabei wird auch immer öfter direkt eine [Containerlösung wie CaaS](#) oder Managed Kubernetes den etablierten virtuellen Maschinen vorgezogen. Dennoch schließen sich beide Konzepte nicht gegenseitig aus. Vielmehr ergänzen sich die Geschwindigkeit leichtgewichtiger Container und die Sicherheit virtueller Maschinen und in Kombination profitiert man von den Vorteilen beider Konzepte.

Passende Produkte



Managed Kubernetes

jetzt informieren

**Wir machen
Cloud Provider**



© Copyright 2023, gridscale GmbH

[Datenschutzerklärung](#) [AGB](#) [Impressum & Kontakt](#)